3DGEngine
Game UI Setup Guide

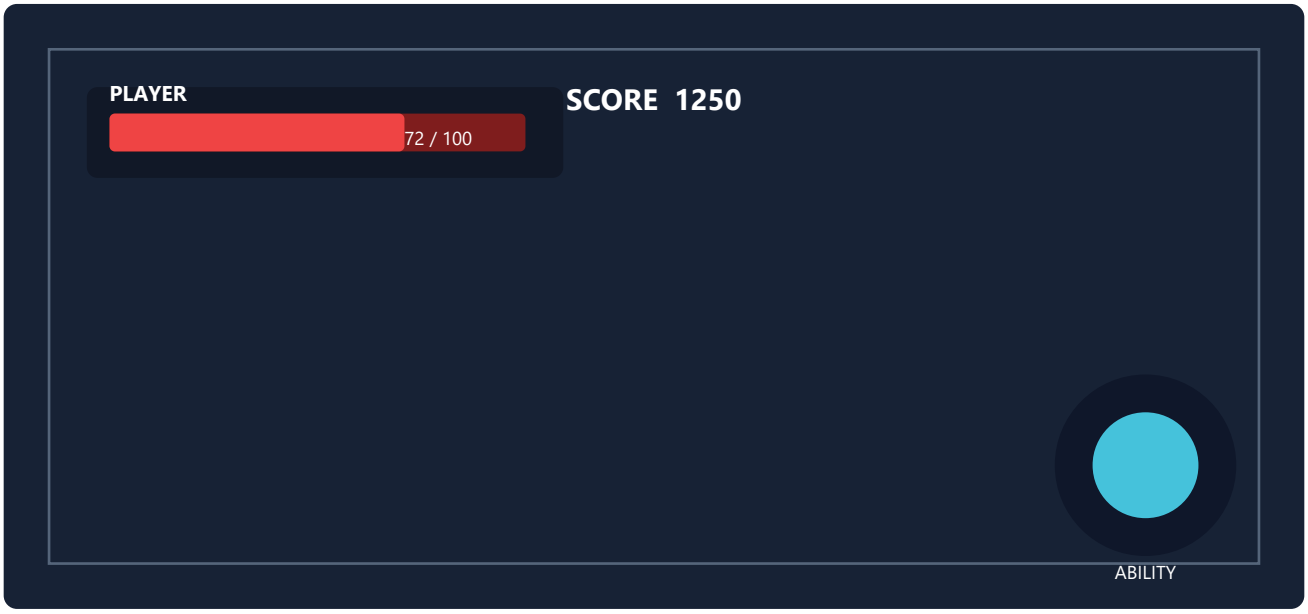
HUD authoring, health bars, menus, bindings, images, and script-driven values

For editor Play mode and the standalone runtime player

Build responsive game HUDs and menus with reusable .hud assets.

Contents

Current engine scope
The UI is a flat, data-driven HUD document. It renders Panels, Text, Bars, Buttons, and Images. Widgets use nine screen anchors plus design-pixel offsets. The scene references the saved .hud asset.



1. How the HUD system works

A HudDocument stores widgets and a 1920 x 1080 design size. Each frame, the editor or player builds a HudContext containing health, named values, mouse state, and texture lookup. DrawHud resolves bindings and renders the widgets in list order.

Part	Purpose
.hud asset	Reusable widget layout saved by the HUD Editor.
Scene HUD reference	Environment setting that tells Play mode which asset to load.
HudContext	Live health, score, time, state, message, and cursor values.
Binding	Connects a Text or Bar widget to a live value.
Button action	None, Exit Play, Restart Play, or Emit Event.

Draw order

Widgets are drawn in document order. Put large background Panels first, then Images and Bars, then Text and Buttons. Later widgets appear on top.

> **No hierarchy layout yet**
> The parent field is reserved, but current positioning is flat. Moving a Panel does not move other widgets automatically.

2. Create and assign a HUD asset

- Open Panels > HUD Editor.
- Enter a path such as Content/UI/GameHUD.hud.
- Click New, then add widgets with Panel, Text, Bar, Button, or Image.
- Save the document, then click Use in Scene.
- Save the scene and export it again for the standalone player.

```
Recommended content layout
Content/
  UI/
    GameHUD.hud
    MainMenu.hud
    PauseMenu.hud
    Images/
      staff_icon.png
      panel_frame.png
```

3. Layout, anchors, and scaling

Choose the anchor that represents the widget's intended screen region. Offset is measured from that anchor in design pixels. Center and right anchors automatically subtract half or all of the widget width; middle and bottom anchors do the same vertically.

Anchor	Typical use	Offset direction
Top Left	Health, quest text	Positive X moves right; positive Y moves down.
Top Center	Score, objective banner	Offset is relative to screen center.
Top Right	Mini-map, resources	Negative X moves left from right edge.
Center	Main menu, prompts	Offset is relative to exact center.
Bottom Left	Chat, item notifications	Negative Y moves upward.
Bottom Center	Ability bar, subtitles	Centered horizontally.
Bottom Right	Ammo, skill icons	Negative X/Y moves inward.

Resolution behavior

The runtime scales widget size and offsets using screen height divided by design height. Anchors keep edge and center relationships stable across resolutions. Test 16:9, ultrawide, and a small window.

> **Practical rule**
> Anchor by meaning, not by current appearance. A health bar belongs at Top Left even if you drag it near the center during experimentation.

4. Widgets, images, and styling

Widget	Important properties	Use
Panel	Color, alpha, size, shader	Backgrounds, darkening overlays, frames.
Text	Text, text scale, color, binding	Labels and live values. Put {} where the bound value should appear.
Bar	Background, fill, binding, min/max	Health, mana, stamina, loading progress.
Button	Text, color, action, event key	Exit, restart, and menu interaction.
Image	Image asset, tint, shader	Logos, icons, portraits, decorative frames.

Images

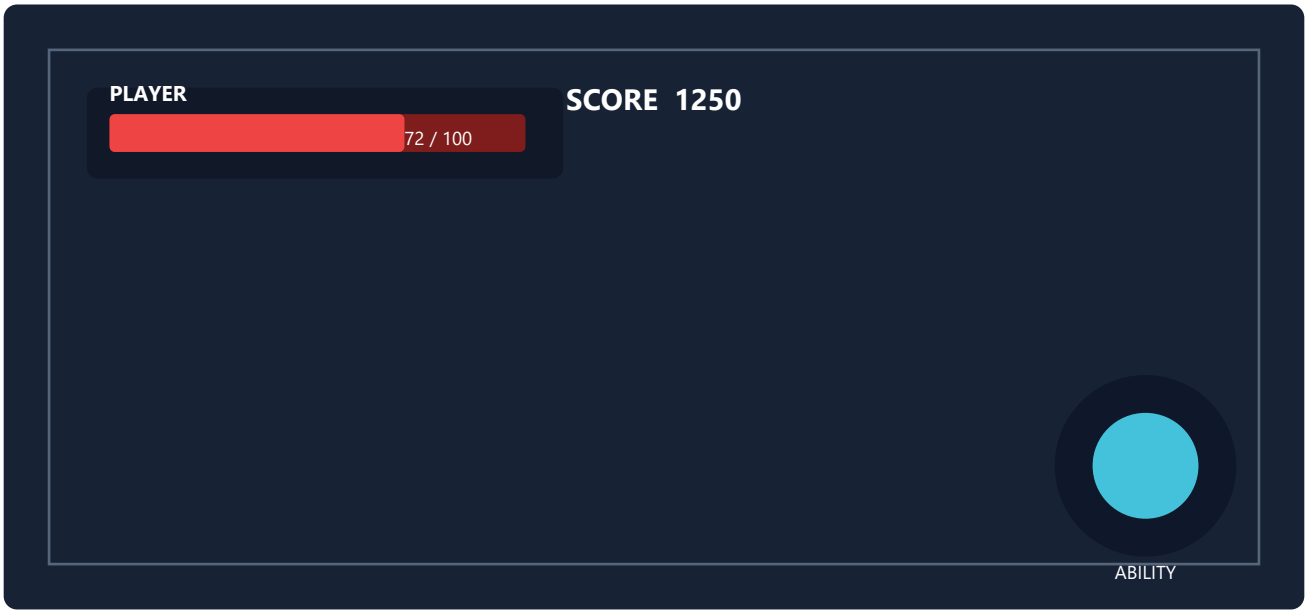
Place PNG/JPG assets under the content folder, choose the Image widget, click Refresh, and select the asset. A white tint preserves original colors. The standalone player resolves image paths relative to the runtime scene directory, so package images with the scene.

Unlit UI shaders

Panel, Bar, Button, and Image widgets can reference an Unlit-domain shader graph. Reflected shader parameters and texture inputs are resolved at runtime. Text widgets do not use custom shaders.

- Use alpha below 1.0 for translucent panels.
- Keep text contrast high over gameplay.
- Use the same size and spacing for related buttons.
- Use duplicate to preserve a consistent visual style.

5. Build a player health HUD



Scene preparation

- Enable Health on the player object and set HP and Max HP.
- Enable Player Controller on that same object when applicable.
- The player runtime binds health widgets to the player; if no player exists, it uses the first Health entity.

Widget recipe

Order	Widget	Settings
1	Panel	Top Left; offset 24,24; size 360,90; dark translucent color.
2	Text	Top Left; offset 42,34; text PLAYER; binding None.
3	Bar	Top Left; offset 42,62; size 300,22; binding Health Fraction.
4	Text	Top Left; offset 245,34; text {}; binding Health Text.

Automatic binding
Health Fraction normalizes HP / Max HP and clamps it to 0..1. Health Text formats the same component as hp/maxHp. No HUD-specific script call is required.

6. Update health from a gameplay script

The HUD reads the Health component every frame. Scripts update the component; the bar and text follow automatically.

```
#include <engine/gameplay/Script.h>
#include <engine/gameplay/GameplayComponents.h>
#include <GLFW/glfw3.h>
#include <algorithm>

class PlayerVitalsScript final : public engine::Script {
public:
    void OnUpdate(float) override {
        auto* health = TryGet<engine::Health>();
        if (!health) return;

        if (WasKeyPressed(GLFW_KEY_H)) {
            health->Damage(GetFieldFloat("testDamage", 15.0f));
        }
        if (WasKeyPressed(GLFW_KEY_J)) {
            health->hp = std::min(
                health->hp + GetFieldFloat("healAmount", 20.0f),
                health->maxHp);
        }
    }
};
```

Damage another object

```
const auto player = FindObject("PlayerStart");
if (IsTriggerTouching(player)) {
    if (auto* health = TryGet<engine::Health>(player)) {
        health->Damage(damagePerSecond * dt);
    }
}
```

Health system timing
Damage reduces HP immediately. The gameplay health system updates alive and justDied during the fixed simulation step. Bindings still display the current HP value each rendered frame.

7. Score, time, state, and messages

The standalone player exposes GameMode values as named HUD bindings. Scripts update GameMode directly.

Bind key	Binding	Meaning
score	Named Float or Named String	Current integer score.
time	Named Float	Elapsed playing time in seconds.
gamestate	Named String	Playing, Paused, Game Over, or Victory.
gamemessage	Named String	Message supplied to Win, Lose, or SetMessage.
fps	Named Float	Current smoothed frame rate.
hp / maxhp	Named Float	Player health values.

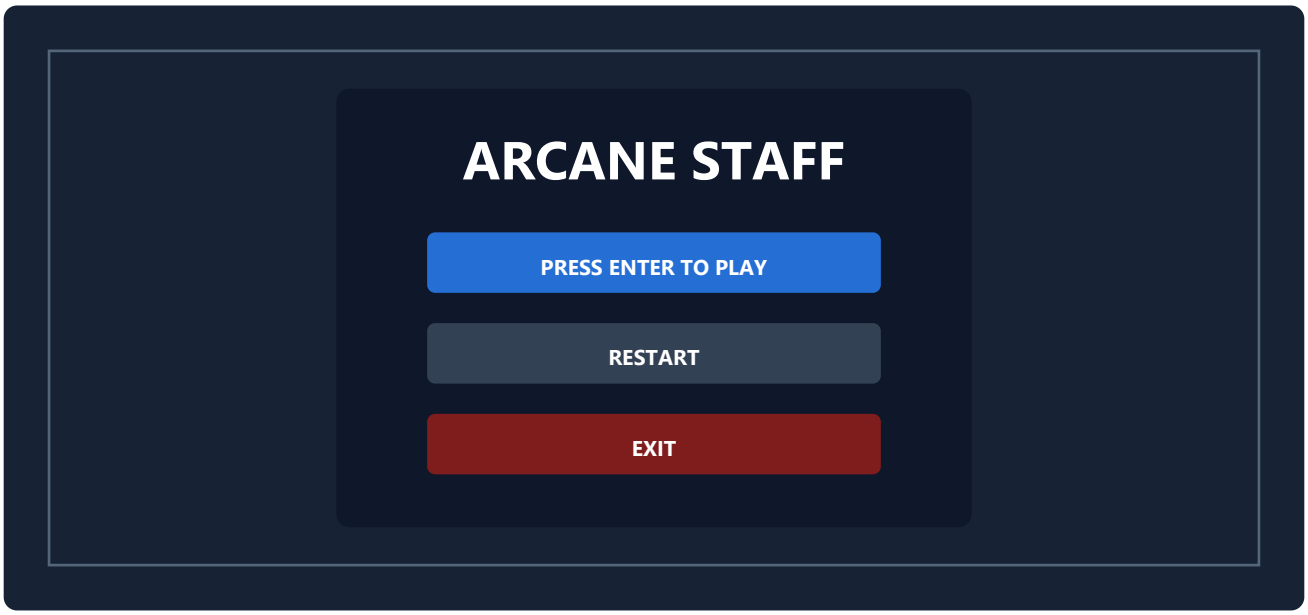
```
#include <engine/gameplay/GameMode.h>

void EnemyRewardScript::OnDestroy() {
    engine::GameMode::Instance().AddScore(
        GetFieldInt("scoreReward", 100));
}

void GoalScript::OnUpdate(float) {
    if (WasTriggerEntered(FindObject("PlayerStart"))) {
        auto& game = engine::GameMode::Instance();
        game.AddScore(500);
        game.Win("The staff has been restored!");
    }
}
```

Text examples

- Text SCORE: {} with Named Float key score.
- Text TIME: {} with Named Float key time.
- Text {} with Named String key gamemessage.
- Bar with Named Float key score, Min 0, Max 5000 for level progress.

Editor preview difference
GameMode bindings are populated by the standalone player. Editor Play currently previews built-in health, hp, maxhp, fps, and editor-pushed values; test final score/state/message behavior in the standalone player.

8. Main menu and pause menu



Main menu asset

- Create Content/UI/MainMenu.hud and assign it to a menu scene.
- Add a full-screen dark Panel, centered logo/title, instructions, and Buttons.
- A menu scene without a Player Controller leaves the cursor available for buttons.
- Set Exit button action to Exit Play. Set Restart button action to Restart Play.

Start game logic

The current button action list does not include Load Scene, and standalone Emit Event is not forwarded to scripts. Use a keyboard start prompt in the current engine, or add a future button-to-scene action.

```
#include <GLFW/glfw3.h>

void MainMenuScript::OnUpdate(float) {
    if (WasKeyPressed(GLFW_KEY_ENTER)) {
        RequestSceneLoad("Levels/Opening.3dgscene");
    }
}
```

Pause menu

In the standalone player, P pauses and frees the cursor; P again resumes. A pause HUD can include Restart and Exit buttons. Buttons only react while the cursor is free. With a player controller, that normally means paused mode.

> **Current limitation**
> Widget visibility is authored, not script-controlled. A separate automatic pause overlay or per-widget visibility API is not yet exposed to scripts. Design one HUD that remains acceptable during play and pause, or use separate scenes/assets.

9. Game over, victory, and other screens

GameMode automatically loses when the player Health dies unless loseOnPlayerDeath is disabled. Scripts can call Win, Lose, Pause, Resume, SetScore, AddScore, and SetMessage.

```
auto& game = engine::GameMode::Instance();

if (bossDefeated) {
    game.Win("Victory - the realm is safe.");
}
if (outOfTime) {
    game.Lose("The ritual was completed.");
}
```

Suggested HUD composition

Screen	Widgets	Binding
Gameplay	Health bar, health text, score, timer, ability icons	Health + score + time
Menu	Full-screen panel, logo image, instructions, exit/restart buttons	Static
Game over	Dark panel, GAME OVER title, message, score, restart/exit	gamemessage + score
Victory	Tinted panel, VICTORY title, message, score	gamemessage + score
Boss fight	Boss name, Named Float bar, phase text	Requires a future arbitrary HUD-value script API

Visibility limitation
Game state and message text update live, but automatic conditional widget visibility is not implemented. Game-over-specific panels will remain visible if included in the same HUD. A future visibility binding is the clean engine extension.

Other useful UI designs

- Objective banner using static Text or gamemessage.
- Score and timer challenge HUD.
- Ability icons using Image widgets.
- Loading/progress bar using Named Float when supplied by the host.
- Cinematic letterbox using two translucent Panels.

10. Testing, packaging, and troubleshooting

Problem	Cause	Fix
HUD does not appear	Scene has no HUD asset reference.	Open HUD Editor, load asset, click Use in Scene, save/export.
Health stays full	Wrong binding or no Health component.	Use Health Fraction; attach Health to player.
Text ignores value	Binding is None or no {} placement.	Choose binding; use {} inside text.
Image is blank	Asset path cannot resolve.	Package image beside scene/content and use Refresh.
Button cannot click	Cursor is captured.	Pause with P, or use a menu scene without a player.
Score works only in player	Editor Play does not populate GameMode bindings.	Validate GameMode UI in standalone runtime.
Widgets shift	Wrong anchor for intended region.	Choose semantic anchor and retest resolutions.
New HUD changes absent	Old runtime scene/HUD was packaged.	Save .hud, export scene, replace packaged content.

Final checklist

- Background widgets come before foreground widgets.
- Every live Text/Bar has the correct binding and key.
- Health is on the controlled player.
- Buttons have an action and are tested with a free cursor.
- Images and .hud files are included in the package.
- The scene's Environment points to the correct HUD asset.
- Layouts are checked at 1280x720, 1920x1080, and ultrawide.

11. Binding and action reference

Binding	Text behavior	Bar behavior
None	Shows authored text.	Shows full bar.
Health Fraction	Shows authored text.	HP / Max HP, clamped 0..1.
Health Text	Formats hp/maxHp; {} substitutes value.	Not intended for bars.
Named Float	Formats numeric key; {} substitutes value.	Normalizes key between Min and Max.
Named String	Shows string key; {} substitutes value.	Not intended for bars.

Button actions

Action	Editor Play	Standalone player
None	No action.	No action.
Exit Play	Leaves Play mode.	Closes player.
Restart Play	Restarts Play mode.	Reloads current scene.
Emit Event	Writes editor HUD float event key.	Currently ignored by standalone player.

Recommended script-to-UI paths

- Health bar/text: modify engine::Health on the player.
- Score: GameMode::AddScore or SetScore.
- Timer: bind time; GameMode advances it while Playing.
- State/message: GameMode::Win, Lose, Pause, Resume, or SetMessage.
- Scene start: RequestSceneLoad from keyboard/trigger script.
- Custom mana/ammo/boss bars: requires a future script HUD-value service.

Source of truth
HUD data: engine/include/engine/ui/Hud.h. Rendering and bindings: engine/src/ui/Hud.cpp. Editor authoring: editor/src/HudEditorPanel.cpp. Runtime values: player/src/RuntimePlayerApp.cpp.

End of guide